InternNova Data Analytics Internship

Week 2 Assignment: NumPy & Pandas for Data Analytics

Submitted by: Brojo Mohan Dutta

Duration: 1 Week (7 Days) | Total Marks: 100

Python source code for all tasks

Screenshots of program outputs

Brief explanation of each task

Dataset (used for Tasks 5–10)

employees.csv

emp_id	name	department	age	salary	city	join_year
101	Aarav Sharma	Sales	29	42000	Kolkata	2021
102	Priya Nair	Marketing	34	55000	Mumbai	2019
103	Rohan Das	Sales	26		Kolkata	2022
104	Sneha Roy	IT	31	68000	Bangalore	2020
105	Karan Mehta	IT		72000	Bangalore	2018
106	Anjali Gupta	Marketing	28	51000	Delhi	2021
107	Vikram Singh	Sales	45		Mumbai	2015
108	Neha Verma	IT	27	65000	Bangalore	2022
109	Arjun Reddy	Finance	38	60000	Chennai	2017
110	Divya Iyer	Finance		58000	Chennai	2019
111	Rahul Kapoor	Sales	33	47000	Kolkata	2020
112	Pooja Bansal	Marketing		53000	Delhi	2021
113	Manish Yadav	IT	29		Bangalore	2020
114	Kavita Joshi	Finance	41	63000	Chennai	2016
115	Suresh Pillai	Sales	36	49000	Mumbai	2018

departments.csv

department	head	location
Sales	Amit Malhotra	Kolkata
Marketing	Ritu Chawla	Delhi
IT	Sanjay Kumar	Bangalore
Finance	Meera Nair	Chennai

Objective

This report develops practical proficiency in NumPy and Pandas, the core Python libraries for data analytics. It covers array creation, indexing, reshaping, and statistical computation with NumPy; and data loading, inspection, filtering, missing-value handling, merging/concatenation, GroupBy aggregation, and pivot tables with Pandas — culminating in a mini analysis project that derives insights from an employee dataset.

Task 1: NumPy Introduction & Arrays

Explanation: Creates a 1D NumPy array of 10 numbers and inspects its shape, size, and dtype; also creates a 1D and 2D array for comparison.

Python Code:

```
import numpy as np

arr = np.array([12, 45, 23, 67, 34, 89, 15, 56, 78, 90])
print("Array:", arr)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data Type:", arr.dtype)

arr_1d = np.array([1, 2, 3, 4, 5])
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
print("1D Array:\n", arr_1d)
print("2D Array:\n", arr_2d)
```

Output Screenshot:

```
Array: [12 45 23 67 34 89 15 56 78 90]
Shape: (10,)
Size: 10
Data Type: int64
1D Array:
 [1 2 3 4 5]
2D Array:
 [[1 2 3]
 [4 5 6]]
```

Task 2: NumPy Indexing, Slicing & Reshaping

Explanation: Demonstrates indexing/slicing on a 1D array, row/column access on a 2D array, and reshaping a flat array into a 2×4 matrix..

Python Code:

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
print("Element at index 3:", arr[3])
print("Sliced array (index 2 to 5):", arr[2:6])

arr_2d = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print("2D Array:\n", arr_2d)
```

```
print("Row 1:", arr_2d[1])
print("Column 2:", arr_2d[:, 2])

reshaped = arr.reshape(2, 4)
print("Original array:", arr)
print("Reshaped array (2x4):\n", reshaped)
```

Output Screenshot:

```
Element at index 3: 40
Sliced array (index 2 to 5): [30 40 50 60]
2D Array:
 [[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Row 1: [5 6 7 8]
Column 2: [ 3  7 11]
Original array: [10 20 30 40 50 60 70 80]
Reshaped array (2x4):
 [[10 20 30 40]
 [50 60 70 80]]
```

Task 3: NumPy Mathematical & Statistical Operations

Explanation: Performs elementwise arithmetic on two arrays, then computes mean, median, min, max, std deviation, and sum.

Python Code:

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
b = np.array([1, 2, 3, 4, 5])

print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)

print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Standard Deviation:", np.std(a))
print("Sum:", np.sum(a))
```

Output Screenshot:

```
Addition: [11 22 33 44 55]
Subtraction: [ 9 18 27 36 45]
Multiplication: [ 10  40  90 160 250]
Division: [10. 10. 10. 10. 10.]
Mean: 30.0
Median: 30.0
Minimum: 10
Maximum: 50
Standard Deviation: 14.142135623730951
Sum: 150
```

Task 4: Pandas Series & Data Frame

Explanation: Builds a Series, then a DataFrame of employee info; inspects columns/index and adds a computed bonus column.

Python Code:

```
import pandas as pd

s = pd.Series([25, 30, 35, 40], index=["a", "b", "c", "d"])
print("Pandas Series:\n", s)

data = {
    "name": ["Aarav Sharma", "Priya Nair", "Rohan Das"],
    "department": ["Sales", "Marketing", "Sales"],
    "salary": [42000, 55000, 48000]
}
df = pd.DataFrame(data)
print("DataFrame:\n", df)
print("Column Names:", df.columns.tolist())
print("Index:", df.index.tolist())

df["bonus"] = df["salary"] * 0.10
print("Updated DataFrame with bonus column:\n", df)
```

Output Screenshot:

```
Pandas Series:
 a    25
b    30
c    35
d    40
dtype: int64
DataFrame:
            name department  salary
0  Aarav Sharma      Sales   42000
1    Priya Nair  Marketing   55000
2     Rohan Das      Sales   48000
Column Names: ['name', 'department', 'salary']
Index: [0, 1, 2]
Updated DataFrame with bonus column:
            name department  salary   bonus
0  Aarav Sharma      Sales   42000  4200.0
1    Priya Nair  Marketing   55000  5500.0
2     Rohan Das      Sales   48000  4800.0
```

Task 5: Reading & Inspecting Data

Explanation: Loads employees.csv and inspects it with head(), tail(), shape, columns, dtypes, info(), and describe().

Python Code:

```
import pandas as pd

df = pd.read_csv("employees.csv")
```

```
print("First 5 rows:\n", df.head())
print("Last 5 rows:\n", df.tail())
print("Shape (rows, columns):", df.shape)
print("Column Names:", df.columns.tolist())
print("Data Types:\n", df.dtypes)
print("Info:")
df.info()
print("Describe:\n", df.describe())
```

Output Screenshot:

```
First 5 rows:
    emp_id          name department   age   salary       city  join_year
0      101  Aarav Sharma      Sales  29.0  42000.0    Kolkata       2021
1      102    Priya Nair  Marketing  34.0  55000.0     Mumbai       2019
2      103     Rohan Das      Sales  26.0      NaN    Kolkata       2022
3      104     Sneha Roy         IT  31.0  68000.0  Bangalore       2020
4      105   Karan Mehta         IT   NaN  72000.0  Bangalore       2018
Last 5 rows:
     emp_id           name department   age   salary       city  join_year
10      111   Rahul Kapoor      Sales  33.0  47000.0    Kolkata       2020
11      112   Pooja Bansal  Marketing   NaN  53000.0      Delhi       2021
12      113   Manish Yadav         IT  29.0      NaN  Bangalore       2020
13      114   Kavita Joshi    Finance  41.0  63000.0    Chennai       2016
14      115  Suresh Pillai      Sales  36.0  49000.0     Mumbai       2018
Shape (rows, columns): (15, 7)
Column Names: ['emp_id', 'name', 'department', 'age', 'salary', 'city', 'join_year']
Data Types:
 emp_id          int64
name            object
department      object
age            float64
salary         float64
city            object
join_year        int64
dtype: object
Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   emp_id      15 non-null     int64
 1   name        15 non-null     object
 2   department  15 non-null     object
 3   age         12 non-null     float64
 4   salary      12 non-null     float64
 5   city        15 non-null     object
 6   join_year   15 non-null     int64
dtypes: float64(2), int64(2), object(3)
memory usage: 972.0+ bytes
```

```
Describe:
            emp_id        age        salary    join_year
count   15.000000  12.000000     12.000000    15.000000
mean   108.000000  33.083333  56916.666667  2019.266667
std      4.472136   5.946096   9049.945588     2.120198
min    101.000000  26.000000  42000.000000  2015.000000
25%    104.500000  28.750000  50500.000000  2018.000000
50%    108.000000  32.000000  56500.000000  2020.000000
75%    111.500000  36.500000  63500.000000  2021.000000
max    115.000000  45.000000  72000.000000  2022.000000
```

Task 6: Selecting, Filtering & Sorting Data

Explanation: Selects specific columns and rows, applies single and multiple filter conditions, and sorts the data in ascending and descending order by salary.

Python Code:

```
import pandas as pd

df = pd.read_csv("employees.csv")

print("Selected columns (name, department, salary):\n", df[["name", "department",
"salary"]])

print("Selected rows (index 0 to 4):\n", df.iloc[0:5])

print("Filter: salary > 55000:\n", df[df["salary"] > 55000])

print("Multiple conditions: department == 'Sales' AND salary > 45000:\n",
      df[(df["department"] == "Sales") & (df["salary"] > 45000)])

print("Sorted ascending by salary:\n", df.sort_values("salary"))
print("Sorted descending by salary:\n", df.sort_values("salary", ascending=False))
```

Output Screenshot:

```
Selected columns (name, department, salary):
              name department   salary
0    Aarav Sharma      Sales  42000.0
1      Priya Nair  Marketing  55000.0
2       Rohan Das      Sales      NaN
3       Sneha Roy         IT  68000.0
4     Karan Mehta         IT  72000.0
5    Anjali Gupta  Marketing  51000.0
6    Vikram Singh      Sales      NaN
7      Neha Verma         IT  65000.0
8     Arjun Reddy    Finance  60000.0
9      Divya Iyer    Finance  58000.0
10   Rahul Kapoor      Sales  47000.0
11   Pooja Bansal  Marketing  53000.0
12   Manish Yadav         IT      NaN
13   Kavita Joshi    Finance  63000.0
14  Suresh Pillai      Sales  49000.0
```

Selected rows (index 0 to 4):

	emp_id	name	department	age	salary	city	join_year
0	101	Aarav Sharma	Sales	29.0	42000.0	Kolkata	2021
1	102	Priya Nair	Marketing	34.0	55000.0	Mumbai	2019
2	103	Rohan Das	Sales	26.0	NaN	Kolkata	2022
3	104	Sneha Roy	IT	31.0	68000.0	Bangalore	2020
4	105	Karan Mehta	IT	NaN	72000.0	Bangalore	2018

Filter: salary > 55000:

	emp_id	name	department	age	salary	city	join_year
3	104	Sneha Roy	IT	31.0	68000.0	Bangalore	2020
4	105	Karan Mehta	IT	NaN	72000.0	Bangalore	2018
7	108	Neha Verma	IT	27.0	65000.0	Bangalore	2022
8	109	Arjun Reddy	Finance	38.0	60000.0	Chennai	2017
9	110	Divya Iyer	Finance	NaN	58000.0	Chennai	2019
13	114	Kavita Joshi	Finance	41.0	63000.0	Chennai	2016

Multiple conditions: department == 'Sales' AND salary > 45000:

	emp_id	name	department	age	salary	city	join_year
10	111	Rahul Kapoor	Sales	33.0	47000.0	Kolkata	2020
14	115	Suresh Pillai	Sales	36.0	49000.0	Mumbai	2018

Sorted ascending by salary:

	emp_id	name	department	age	salary	city	join_year
0	101	Aarav Sharma	Sales	29.0	42000.0	Kolkata	2021
10	111	Rahul Kapoor	Sales	33.0	47000.0	Kolkata	2020
14	115	Suresh Pillai	Sales	36.0	49000.0	Mumbai	2018
5	106	Anjali Gupta	Marketing	28.0	51000.0	Delhi	2021
11	112	Pooja Bansal	Marketing	NaN	53000.0	Delhi	2021
1	102	Priya Nair	Marketing	34.0	55000.0	Mumbai	2019
9	110	Divya Iyer	Finance	NaN	58000.0	Chennai	2019
8	109	Arjun Reddy	Finance	38.0	60000.0	Chennai	2017
13	114	Kavita Joshi	Finance	41.0	63000.0	Chennai	2016
7	108	Neha Verma	IT	27.0	65000.0	Bangalore	2022
3	104	Sneha Roy	IT	31.0	68000.0	Bangalore	2020
4	105	Karan Mehta	IT	NaN	72000.0	Bangalore	2018
2	103	Rohan Das	Sales	26.0	NaN	Kolkata	2022
6	107	Vikram Singh	Sales	45.0	NaN	Mumbai	2015
12	113	Manish Yadav	IT	29.0	NaN	Bangalore	2020

Sorted descending by salary:

	emp_id	name	department	age	salary	city	join_year
4	105	Karan Mehta	IT	NaN	72000.0	Bangalore	2018
3	104	Sneha Roy	IT	31.0	68000.0	Bangalore	2020
7	108	Neha Verma	IT	27.0	65000.0	Bangalore	2022
13	114	Kavita Joshi	Finance	41.0	63000.0	Chennai	2016
8	109	Arjun Reddy	Finance	38.0	60000.0	Chennai	2017
9	110	Divya Iyer	Finance	NaN	58000.0	Chennai	2019
1	102	Priya Nair	Marketing	34.0	55000.0	Mumbai	2019
11	112	Pooja Bansal	Marketing	NaN	53000.0	Delhi	2021
5	106	Anjali Gupta	Marketing	28.0	51000.0	Delhi	2021
14	115	Suresh Pillai	Sales	36.0	49000.0	Mumbai	2018
10	111	Rahul Kapoor	Sales	33.0	47000.0	Kolkata	2020
0	101	Aarav Sharma	Sales	29.0	42000.0	Kolkata	2021
2	103	Rohan Das	Sales	26.0	NaN	Kolkata	2022
6	107	Vikram Singh	Sales	45.0	NaN	Mumbai	2015
12	113	Manish Yadav	IT	29.0	NaN	Bangalore	2020

Task 7: Handling Missing Values

Explanation: Identifies missing values with isnull(), counts them per column, drops rows containing them, and fills them using mean/median. Handling missing data matters because unaddressed nulls skew statistical summaries, break aggregations, and can cause errors in downstream models — cleaning ensures accurate, reliable analysis.

Python Code:

```
import pandas as pd

df = pd.read_csv("employees.csv")

print("Missing values (True/False):\n", df.isnull())
print("Missing values count per column:\n", df.isnull().sum())

df_dropped = df.dropna()
print("Dataset after dropping rows with missing values:\n", df_dropped)

df_filled = df.copy()
df_filled["age"] = df_filled["age"].fillna(df_filled["age"].mean())
df_filled["salary"] = df_filled["salary"].fillna(df_filled["salary"].median())
print("Dataset after filling missing values (age->mean, salary->median):\n", df_filled)
```

Output Screenshot:

```
Missing values (True/False):
     emp_id   name  department    age  salary   city  join_year
0    False  False       False  False   False  False      False
1    False  False       False  False   False  False      False
2    False  False       False  False    True  False      False
3    False  False       False  False   False  False      False
4    False  False       False   True   False  False      False
5    False  False       False  False   False  False      False
6    False  False       False  False    True  False      False
7    False  False       False  False   False  False      False
8    False  False       False  False   False  False      False
9    False  False       False   True   False  False      False
10   False  False       False  False   False  False      False
11   False  False       False   True   False  False      False
12   False  False       False  False    True  False      False
13   False  False       False  False   False  False      False
14   False  False       False  False   False  False      False
Missing values count per column:
 emp_id        0
name           0
department     0
age            3
salary         3
city           0
join_year      0
dtype: int64
```

```
Dataset after dropping rows with missing values:
     emp_id           name department   age   salary       city  join_year
0       101   Aarav Sharma      Sales  29.0  42000.0    Kolkata       2021
1       102     Priya Nair  Marketing  34.0  55000.0     Mumbai       2019
3       104      Sneha Roy         IT  31.0  68000.0  Bangalore       2020
5       106   Anjali Gupta  Marketing  28.0  51000.0      Delhi       2021
7       108     Neha Verma         IT  27.0  65000.0  Bangalore       2022
8       109    Arjun Reddy    Finance  38.0  60000.0    Chennai       2017
10      111   Rahul Kapoor      Sales  33.0  47000.0    Kolkata       2020
13      114   Kavita Joshi    Finance  41.0  63000.0    Chennai       2016
14      115  Suresh Pillai      Sales  36.0  49000.0     Mumbai       2018
Dataset after filling missing values (age->mean, salary->median):
     emp_id           name department        age   salary       city  join_year
0       101   Aarav Sharma      Sales  29.000000  42000.0    Kolkata       2021
1       102     Priya Nair  Marketing  34.000000  55000.0     Mumbai       2019
2       103      Rohan Das      Sales  26.000000  56500.0    Kolkata       2022
3       104      Sneha Roy         IT  31.000000  68000.0  Bangalore       2020
4       105    Karan Mehta         IT  33.083333  72000.0  Bangalore       2018
5       106   Anjali Gupta  Marketing  28.000000  51000.0      Delhi       2021
6       107   Vikram Singh      Sales  45.000000  56500.0     Mumbai       2015
7       108     Neha Verma         IT  27.000000  65000.0  Bangalore       2022
8       109    Arjun Reddy    Finance  38.000000  60000.0    Chennai       2017
9       110     Divya Iyer    Finance  33.083333  58000.0    Chennai       2019
10      111   Rahul Kapoor      Sales  33.000000  47000.0    Kolkata       2020
11      112   Pooja Bansal  Marketing  33.083333  53000.0      Delhi       2021
12      113   Manish Yadav         IT  29.000000  56500.0  Bangalore       2020
13      114   Kavita Joshi    Finance  41.000000  63000.0    Chennai       2016
14      115  Suresh Pillai      Sales  36.000000  49000.0     Mumbai       2018
```

Task 8: Merge, Concatenate, GroupBy & Pivot Table

Explanation: Merges employees.csv with departments.csv on the common department column, concatenates two new hires onto the employee list, groups salary by department (sum/mean/count/min/max), and builds a pivot table of average salary by department and city.

Python Code:

```
import pandas as pd

employees = pd.read_csv("employees.csv")
departments = pd.read_csv("departments.csv")

# Merge
merged_df = pd.merge(employees, departments, on="department", how="left")
print("Merged DataFrame (employees + departments):\n", merged_df)

# Concatenate
new_hires = pd.DataFrame({
    "emp_id": [116, 117],
    "name": ["Tanya Sen", "Farhan Ali"],
    "department": ["Sales", "IT"],
    "age": [24, 30],
    "salary": [44000, 66000],
    "city": ["Kolkata", "Bangalore"],
```

```
    "join_year": [2023, 2023]
})
concatenated_df = pd.concat([employees, new_hires], ignore_index=True)
print("Concatenated DataFrame (original + new hires):\n", concatenated_df)

# GroupBy
grouped = employees.groupby("department")["salary"].agg(["sum", "mean", "count", "min",
"max"])
print("GroupBy department -> salary aggregates:\n", grouped)

# Pivot Table
pivot = pd.pivot_table(employees, values="salary", index="department", columns="city",
aggfunc="mean")
print("Pivot Table (avg salary by department & city):\n", pivot)
```

Output Screenshot:

```
Merged DataFrame (employees + departments):
     emp_id           name department   age   salary       city  join_year  \
0       101   Aarav Sharma      Sales  29.0  42000.0    Kolkata       2021
1       102     Priya Nair  Marketing  34.0  55000.0     Mumbai       2019
2       103      Rohan Das      Sales  26.0      NaN    Kolkata       2022
3       104      Sneha Roy         IT  31.0  68000.0  Bangalore       2020
4       105    Karan Mehta         IT   NaN  72000.0  Bangalore       2018
5       106   Anjali Gupta  Marketing  28.0  51000.0      Delhi       2021
6       107   Vikram Singh      Sales  45.0      NaN     Mumbai       2015
7       108     Neha Verma         IT  27.0  65000.0  Bangalore       2022
8       109    Arjun Reddy    Finance  38.0  60000.0    Chennai       2017
9       110     Divya Iyer    Finance   NaN  58000.0    Chennai       2019
10      111   Rahul Kapoor      Sales  33.0  47000.0    Kolkata       2020
11      112   Pooja Bansal  Marketing   NaN  53000.0      Delhi       2021
12      113   Manish Yadav         IT  29.0      NaN  Bangalore       2020
13      114   Kavita Joshi    Finance  41.0  63000.0    Chennai       2016
14      115  Suresh Pillai      Sales  36.0  49000.0     Mumbai       2018

             head   location
0   Amit Malhotra    Kolkata
1     Ritu Chawla      Delhi
2   Amit Malhotra    Kolkata
3    Sanjay Kumar  Bangalore
4    Sanjay Kumar  Bangalore
5     Ritu Chawla      Delhi
6   Amit Malhotra    Kolkata
7    Sanjay Kumar  Bangalore
8      Meera Nair    Chennai
9      Meera Nair    Chennai
10  Amit Malhotra    Kolkata
11    Ritu Chawla      Delhi
12   Sanjay Kumar  Bangalore
13     Meera Nair    Chennai
14  Amit Malhotra    Kolkata
```

```
Concatenated DataFrame (original + new hires):
     emp_id           name department   age   salary       city  join_year
0       101   Aarav Sharma      Sales  29.0  42000.0    Kolkata       2021
1       102     Priya Nair  Marketing  34.0  55000.0     Mumbai       2019
2       103      Rohan Das      Sales  26.0      NaN    Kolkata       2022
3       104      Sneha Roy         IT  31.0  68000.0  Bangalore       2020
4       105    Karan Mehta         IT   NaN  72000.0  Bangalore       2018
5       106   Anjali Gupta  Marketing  28.0  51000.0      Delhi       2021
6       107   Vikram Singh      Sales  45.0      NaN     Mumbai       2015
7       108     Neha Verma         IT  27.0  65000.0  Bangalore       2022
8       109    Arjun Reddy    Finance  38.0  60000.0    Chennai       2017
9       110     Divya Iyer    Finance   NaN  58000.0    Chennai       2019
10      111   Rahul Kapoor      Sales  33.0  47000.0    Kolkata       2020
11      112   Pooja Bansal  Marketing   NaN  53000.0      Delhi       2021
12      113   Manish Yadav         IT  29.0      NaN  Bangalore       2020
13      114   Kavita Joshi    Finance  41.0  63000.0    Chennai       2016
14      115  Suresh Pillai      Sales  36.0  49000.0     Mumbai       2018
15      116      Tanya Sen      Sales  24.0  44000.0    Kolkata       2023
16      117     Farhan Ali         IT  30.0  66000.0  Bangalore       2023
GroupBy department -> salary aggregates:
                sum          mean  count      min      max
department
Finance     181000.0  60333.333333      3  58000.0  63000.0
IT          205000.0  68333.333333      3  65000.0  72000.0
Marketing   159000.0  53000.000000      3  51000.0  55000.0
Sales       138000.0  46000.000000      3  42000.0  49000.0
Pivot Table (avg salary by department & city):
 city           Bangalore       Chennai    Delhi  Kolkata   Mumbai
department
Finance              NaN  60333.333333      NaN      NaN      NaN
IT          68333.333333           NaN      NaN      NaN      NaN
Marketing            NaN           NaN  52000.0      NaN  55000.0
Sales                NaN           NaN      NaN  44500.0  49000.0
```

Task 9: Exporting Data

Explanation: Cleans missing values in age and salary, exports the processed DataFrame to a new CSV, then re-reads it to verify the export succeeded and contains no missing values.

Python Code:

```
import pandas as pd

df = pd.read_csv("employees.csv")

df["age"] = df["age"].fillna(df["age"].mean())
df["salary"] = df["salary"].fillna(df["salary"].median())

df.to_csv("employees_processed.csv", index=False)

check = pd.read_csv("employees_processed.csv")
print("Exported file preview:\n", check.head())
print("Exported file shape:", check.shape)
print("Missing values after export:\n", check.isnull().sum())
```

Output Screenshot:

```
Exported file preview:
    emp_id          name department        age   salary       city  join_year
0      101  Aarav Sharma      Sales  29.000000  42000.0    Kolkata       2021
1      102    Priya Nair  Marketing  34.000000  55000.0     Mumbai       2019
2      103     Rohan Das      Sales  26.000000  56500.0    Kolkata       2022
3      104     Sneha Roy         IT  31.000000  68000.0  Bangalore       2020
4      105   Karan Mehta         IT  33.083333  72000.0  Bangalore       2018
Exported file shape: (15, 7)
Missing values after export:
 emp_id        0
name           0
department     0
age            0
salary         0
city           0
join_year      0
dtype: int64
```

Task 10: Mini Data Analysis Project

Explanation: End-to-end mini analysis on the Employee Dataset — load, inspect, clean missing values, filter high earners, sort by salary, run GroupBy and a pivot table, derive three insights, then export the cleaned dataset.

Python Code:

```
import pandas as pd

# 1. Load dataset
df = pd.read_csv("employees.csv")

# 2. Data inspection
print("Shape:", df.shape)
df.info()
print("Describe:\n", df.describe())

# 3. Handle missing values
print("Missing values before cleaning:\n", df.isnull().sum())
df["age"] = df["age"].fillna(df["age"].mean()).round().astype(int)
df["salary"] = df["salary"].fillna(df["salary"].median())
print("Missing values after cleaning:\n", df.isnull().sum())

# 4. Selecting & filtering
high_earners = df[df["salary"] > 55000]
print("High earners (salary > 55000):\n", high_earners[["name", "department", "salary"]])

# 5. Sorting
top5_salary = df.sort_values("salary", ascending=False).head(5)
print("Top 5 salaries:\n", top5_salary[["name", "department", "salary"]])

# 6. GroupBy analysis
dept_summary = df.groupby("department")["salary"].agg(["mean", "count"]).round(2)
print("Average salary & headcount by department:\n", dept_summary)

# 7. Pivot table
```

```
pivot = pd.pivot_table(df, values="salary", index="department", columns="city",
aggfunc="mean")
print("Pivot table - avg salary by department & city:\n", pivot)

# 8. Insights
highest_paid_dept = dept_summary["mean"].idxmax()
largest_dept = df["department"].value_counts().idxmax()
print(f"Insight 1: '{highest_paid_dept}' has the highest average salary.")
print(f"Insight 2: '{largest_dept}' has the most employees.")
print(f"Insight 3: Overall average salary is {df['salary'].mean():.2f}.")

# 9. Export cleaned dataset
df.to_csv("employee_analysis_final.csv", index=False)
print("Cleaned dataset exported as 'employee_analysis_final.csv'")
```

Output Screenshot:

```
Shape: (15, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   emp_id      15 non-null     int64
 1   name        15 non-null     object
 2   department  15 non-null     object
 3   age         12 non-null     float64
 4   salary      12 non-null     float64
 5   city        15 non-null     object
 6   join_year   15 non-null     int64
dtypes: float64(2), int64(2), object(3)
memory usage: 972.0+ bytes
Describe:
            emp_id        age        salary    join_year
count   15.000000  12.000000     12.000000    15.000000
mean   108.000000  33.083333  56916.666667  2019.266667
std      4.472136   5.946096   9049.945588     2.120198
min    101.000000  26.000000  42000.000000  2015.000000
25%    104.500000  28.750000  50500.000000  2018.000000
50%    108.000000  32.000000  56500.000000  2020.000000
75%    111.500000  36.500000  63500.000000  2021.000000
max    115.000000  45.000000  72000.000000  2022.000000
Missing values before cleaning:
 emp_id        0
name           0
department     0
age            3
salary         3
city           0
join_year      0
dtype: int64
```

```
Missing values after cleaning:
 emp_id        0
name           0
department     0
age            0
salary         0
city           0
join_year      0
dtype: int64
High earners (salary > 55000):
              name department   salary
2       Rohan Das      Sales  56500.0
3       Sneha Roy         IT  68000.0
4     Karan Mehta         IT  72000.0
6    Vikram Singh      Sales  56500.0
7      Neha Verma         IT  65000.0
8     Arjun Reddy    Finance  60000.0
9      Divya Iyer    Finance  58000.0
12   Manish Yadav         IT  56500.0
13   Kavita Joshi    Finance  63000.0
Top 5 salaries:
              name department   salary
4     Karan Mehta         IT  72000.0
3       Sneha Roy         IT  68000.0
7      Neha Verma         IT  65000.0
13   Kavita Joshi    Finance  63000.0
8     Arjun Reddy    Finance  60000.0
Average salary & headcount by department:
                mean  count
department
Finance     60333.33      3
IT          65375.00      4
Marketing   53000.00      3
Sales       50200.00      5
Pivot table - avg salary by department & city:
 city         Bangalore       Chennai    Delhi  Kolkata   Mumbai
department
Finance            NaN  60333.333333      NaN      NaN      NaN
IT             65375.0           NaN      NaN      NaN      NaN
Marketing          NaN           NaN  52000.0      NaN  55000.0
Sales              NaN           NaN      NaN  48500.0  52750.0
Insight 1: 'IT' has the highest average salary.
Insight 2: 'Sales' has the most employees.
Insight 3: Overall average salary is 56833.33.
Cleaned dataset exported as 'employee_analysis_final.csv'
```

Key Insight:

- **IT** has the highest average salary (₹65,375) among all departments.
- **Sales** has the most employees (5 out of 15).
- Overall average salary across the company is **₹56,833.33**.
- 3 of 15 records (20%) had missing `age` and `salary` values, now cleaned using mean/median imputation.